



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN AN INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN USING GRAPHS, HEAPS AND PRIORITY QUEUES

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

K.Kabilan (192521157)

H.Saravana (192511397)

T.Lokesh waran (192521194)

Under the Supervision Of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105 September

2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply suitable Abstract Data Types for efficient disaster-relief supply management. CO4: Use heaps and priority queues for prioritizing emergency supply requests. CO5: Develop and evaluate an efficient disaster-relief supply chain system under varying loads.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Efficient management of relief supplies, emergency requests and transportation routes for faster delivery to disaster-affected areas.

B. Assignment Problem / Challenge

A disaster-relief organization needs to deliver essential supplies such as food, water, medicines, and emergency equipment from warehouses to several affected areas. Each location may have different requirements, priority levels, available routes, and travel distances.

The challenge is to develop an efficient C-based disaster-relief supply system that represents the transportation network using a graph, finds suitable routes between locations, prioritizes urgent supply requests using a heap and priority queue, and ensures critical areas receive supplies before lower-priority locations.

The system should handle increasing numbers of locations, routes, and relief requests efficiently while reducing delays in emergency supply distribution.

C. Problem Statement

The proposed system manages the delivery of essential disaster-relief supplies from warehouses to affected areas through a connected transportation network. Warehouses, relief centres, hospitals, and affected locations are represented as vertices, while roads and transport routes are represented as weighted edges.

Each relief request includes details such as the destination, required supply, quantity, and urgency level. A max heap-based priority queue is used to process the most urgent requests first, while graph-based route analysis helps find suitable paths for supply transportation.

The system is designed to handle multiple requests, different route distances, limited supplies, and increasing numbers of locations. A complete C program is developed using graphs, heaps, and priority queues, followed by testing and performance analysis with different numbers of locations and relief requests.

D. Requirements and Constraints

Functional Requirements:

- Store information about warehouses, relief centres, hospitals, and affected areas.
- Represent the transportation network using a weighted graph.
- Record and manage relief requests according to their urgency.
- Use a max heap priority queue to process high-urgency requests first.
- Find suitable routes between supply centres and affected locations.
- Display the processing sequence of relief requests.
- Monitor the type and quantity of supplies required at each location.

- Measure system performance for different numbers of locations and requests.

Performance Requirements:

- Graph insertion and adjacency operations should work efficiently for large networks.
- Priority queue insertion and deletion: $O(\log n)$.
- Highest-priority request retrieval: $O(1)$.
- Graph traversal using BFS/DFS: $O(V + E)$.
- Handle growing numbers of locations, routes, and requests with efficient memory usage.

Technical Constraints:

- Implementation language: C.
- Graph representation: Adjacency list.
- Route information: Weighted edges for distance or travel time.
- Priority queue: Max Heap.
- Priority value: Disaster urgency score.
- Performance measurement: `clock()` function.

Security, Reliability and Ethical Constraints:

- Relief requests must be processed according to their assigned urgency levels.
- The system must prevent loss or duplication of supply requests.
- Route details must be stored accurately to prevent incorrect deliveries.
- Disaster-related information must be used only for authorized relief and emergency-management activities.

E. Student Work

1. Problem Understanding and Formulation

The problem is to develop an efficient disaster-relief supply chain system that manages the transportation of essential supplies from warehouses to affected areas. Traditional methods may become less efficient when the number of locations and emergency requests increases.

A graph is used to model the transportation network, with locations as vertices and roads as weighted edges. A max heap-based priority queue is used to organize relief requests based on urgency. This combination helps the system quickly identify critical requests and improve the efficiency of supply distribution.

Expected Outcomes

- Efficient representation of the disaster-relief transportation network.
- Faster identification of suitable routes between supply centres and affected areas.
- Quick identification of the highest-priority relief request.

- Efficient handling of multiple emergency supply requests.
- Performance analysis with increasing numbers of locations, routes, and requests.

Available Data

Field	Example
Source Location	Central Warehouse
Destination	Relief Centre A
Supply Type	Drinking Water
Quantity	500 Units
Urgency Score	95
Route Distance	12 km

Assumptions

- Urgency scores are considered between 0 and 100.
- A higher urgency score represents a more immediate need for relief.
- Locations are treated as vertices in the graph.
- Roads and transportation routes are represented as weighted edges.
- Edge weights indicate distance or travel time.
- Relief requests are managed using a max heap priority queue.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
graph	Represents warehouses, relief centres, hospitals, and affected areas as connected locations.	BFS/DFS: $O(V + E)$
Adjacency List	Stores roads and transportation connections efficiently.	$O(V + E)$
Weighted Graph	Represents the distance or travel time between connected locations.	Depends on graph operation
Max Heap	Organizes relief requests according to their urgency scores.	$O(\log n)$ insertion/deletion n
Priority Queue	Ensures the most urgent relief request is processed first.	$O(1)$ highest-priority retrieval

3.Solution / Design / Methodology

The system models disaster-affected locations and transportation routes using a weighted graph, while relief requests are managed using a max heap priority queue.

The graph is used to identify available routes, while the priority queue ensures that requests with higher urgency are handled first.

The system processes the requests one by one and evaluates its performance with different numbers of locations and relief requests.

Algorithm

1. Initialize the weighted graph and max heap priority queue.
2. Add locations and connecting routes with their distance or travel-time weights.
3. Read each relief request and insert it into the max heap according to its urgency score.
4. Select the highest-priority request and find a suitable route through the graph.
5. Deliver the required supplies, remove the completed request, and continue until all requests are processed.

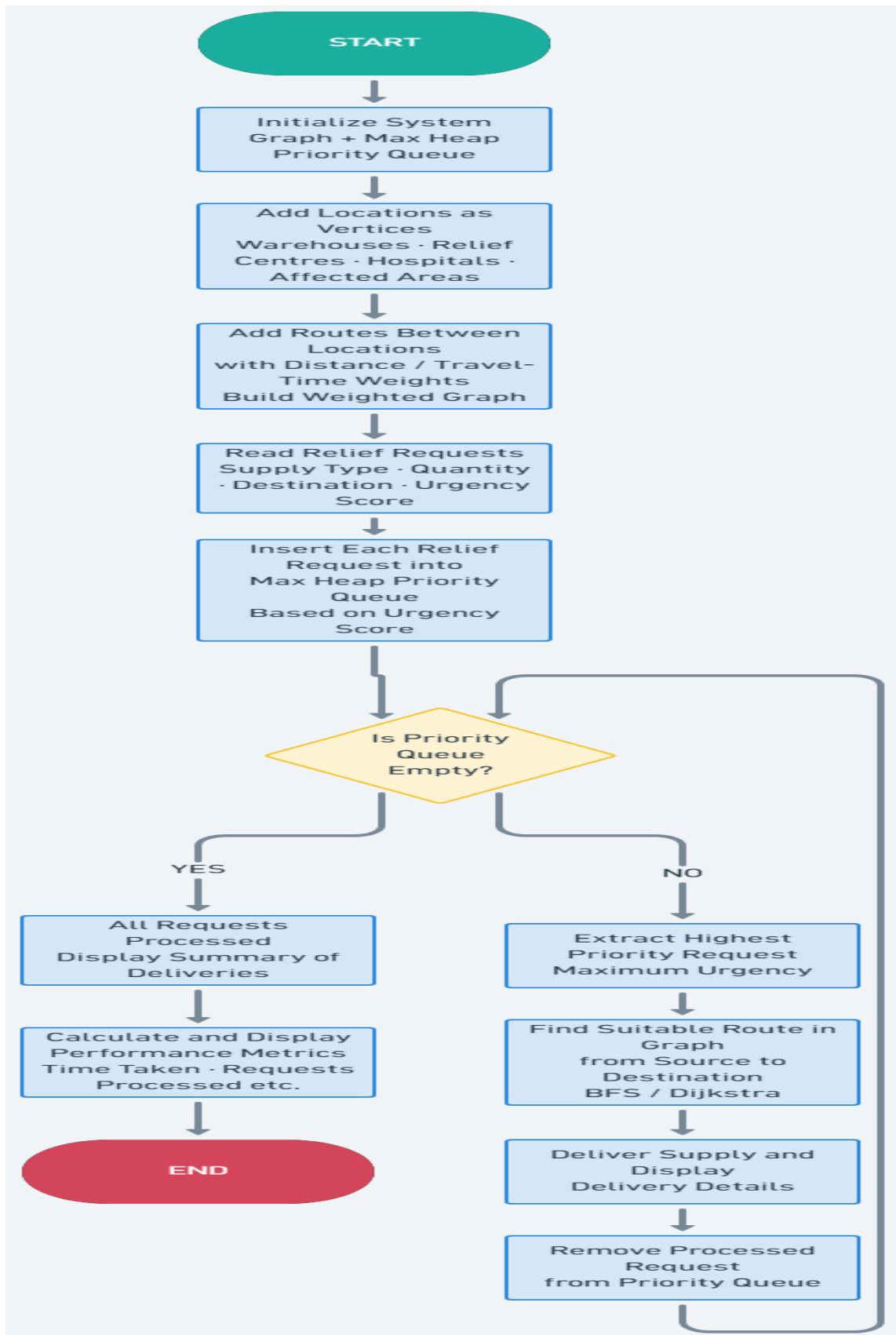


Fig: 1 Flow Chart of disaster relief supply chain system

Implementation Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_NODES 25
#define MAX_REQUESTS 100
#define INF 999999

/* ===== GRAPH STRUCTURE ===== */

typedef struct Route {
    int to;
    int distance;
    struct Route *next;
} Route;

typedef struct {
    char location[50];
    Route *routes;
} Node;

Node network[MAX_NODES];
int nodeCount = 0;

/* Add location */
int createLocation(const char name[])
{
    if (nodeCount >= MAX_NODES)
    {
        printf("Location limit reached.\n");
        return -1;
    }

    strcpy(network[nodeCount].location, name);
    network[nodeCount].routes = NULL;

    return nodeCount++;
}

/* Add one-way route */
void connectRoute(int from, int to, int distance)
{

```



```

Route *newRoute = (Route *)malloc(sizeof(Route));

newRoute->to = to;
newRoute->distance = distance;
newRoute->next = network[from].routes;

network[from].routes = newRoute;
}

/* Add two-way route */
void connectLocations(int a, int b, int distance)
{
    connectRoute(a, b, distance);
    connectRoute(b, a, distance);
}

/* Display transportation network */
void showNetwork()
{
    int i;

    printf("\n===== TRANSPORTATION NETWORK =====\n");

    for (i = 0; i < nodeCount; i++)
    {
        Route *temp = network[i].routes;

        printf("%s : ", network[i].location);

        while (temp != NULL)
        {
            printf("%s [%d km]",
                network[temp->to].location,
                temp->distance);

            if (temp->next != NULL)
                printf(" -> ");

            temp = temp->next;
        }

        printf("\n");
    }
}

```

```

/* Find location index */
int getLocation(const char name[])
{
    int i;

    for (i = 0; i < nodeCount; i++)
    {
        if (strcmp(network[i].location, name) == 0)
            return i;
    }

    return -1;
}

/* ===== RELIEF REQUEST ===== */

typedef struct
{
    int id;
    char source[50];
    char destination[50];
    char item[40];
    int quantity;
    int priority;
} Request;

/* ===== MAX HEAP ===== */

Request priorityHeap[MAX_REQUESTS];
int heapCount = 0;

/* Swap requests */
void swap(Request *a, Request *b)
{
    Request temp = *a;
    *a = *b;
    *b = temp;
}

/* Insert into max heap */
void addRequest(Request r)
{
    int index, parent;

    if (heapCount >= MAX_REQUESTS)

```

```

{
    printf("Request queue is full.\n");
    return;
}

index = heapCount;
priorityHeap[heapCount] = r;
heapCount++;

while (index > 0)
{
    parent = (index - 1) / 2;

    if (priorityHeap[parent].priority >=
        priorityHeap[index].priority)
        break;

    swap(&priorityHeap[parent], &priorityHeap[index]);

    index = parent;
}
}

/* Remove highest-priority request */
Request getTopRequest()
{
    Request top = priorityHeap[0];
    int index = 0;
    int left, right, largest;

    priorityHeap[0] = priorityHeap[heapCount - 1];
    heapCount--;

    while (1)
    {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < heapCount &&
            priorityHeap[left].priority >
            priorityHeap[largest].priority)
        {
            largest = left;
        }
    }
}

```

```

    if (right < heapCount &&
        priorityHeap[right].priority >
        priorityHeap[largest].priority)
    {
        largest = right;
    }

    if (largest == index)
        break;

    swap(&priorityHeap[index],
        &priorityHeap[largest]);

    index = largest;
}

return top;
}

/* Display pending requests */
void showRequests()
{
    int i;

    printf("\n===== PRIORITY QUEUE =====\n");

    if (heapCount == 0)
    {
        printf("No pending requests.\n");
        return;
    }

    for (i = 0; i < heapCount; i++)
    {
        printf("ID: %d | Destination: %s | Supply: %s | "
            "Quantity: %d | Priority: %d\n",
            priorityHeap[i].id,
            priorityHeap[i].destination,
            priorityHeap[i].item,
            priorityHeap[i].quantity,
            priorityHeap[i].priority);
    }
}

```

```
/* ===== DIJKSTRA ALGORITHM ===== */
```

```
void shortestPath(int start, int target)
{
    int distance[MAX_NODES];
    int visited[MAX_NODES];
    int previous[MAX_NODES];

    int i, count;
    int current;
    int minimum;

    for (i = 0; i < nodeCount; i++)
    {
        distance[i] = INF;
        visited[i] = 0;
        previous[i] = -1;
    }

    distance[start] = 0;

    for (count = 0; count < nodeCount; count++)
    {
        current = -1;
        minimum = INF;

        for (i = 0; i < nodeCount; i++)
        {
            if (!visited[i] && distance[i] < minimum)
            {
                minimum = distance[i];
                current = i;
            }
        }

        if (current == -1)
            break;

        visited[current] = 1;

        Route *edge = network[current].routes;

        while (edge != NULL)
        {
            int next = edge->to;
```

```

        if (!visited[next] &&
            distance[current] + edge->distance <
            distance[next])
        {
            distance[next] =
                distance[current] + edge->distance;

            previous[next] = current;
        }

        edge = edge->next;
    }
}

if (distance[target] == INF)
{
    printf("No available route found.\n");
    return;
}

int path[MAX_NODES];
int length = 0;
int position = target;

while (position != -1)
{
    path[length++] = position;
    position = previous[position];
}

printf("Route: ");

for (i = length - 1; i >= 0; i--)
{
    printf("%s", network[path[i]].location);

    if (i != 0)
        printf(" -> ");
}

printf("\nTravel Distance: %d km\n",
    distance[target]);
}

```

```

/* ===== PROCESS REQUESTS ===== */

void dispatchSupplies()
{
    printf("\n===== RELIEF DISPATCH =====\n");

    while (heapCount > 0)
    {
        Request current = getTopRequest();

        printf("\n-----\n");
        printf("Processing Request : %d\n", current.id);
        printf("From          : %s\n", current.source);
        printf("To            : %s\n", current.destination);
        printf("Supply       : %s\n", current.item);
        printf("Quantity     : %d units\n", current.quantity);
        printf("Urgency      : %d\n", current.priority);

        int sourceID = getLocation(current.source);
        int destinationID = getLocation(current.destination);

        if (sourceID == -1 || destinationID == -1)
        {
            printf("Invalid location.\n");
            continue;
        }

        printf("Finding shortest route...\n");

        shortestPath(sourceID, destinationID);

        printf("Status      : SUPPLY DELIVERED\n");
    }

    printf("\nAll relief requests completed successfully.\n");
}

/* ===== PERFORMANCE TEST ===== */

```

```

void runPerformanceTest()
{
    int testSizes[] = {10, 25, 50, 75, 100};
    int testCases = 5;
    int i, j;
}

```

```

printf("\n===== PERFORMANCE ANALYSIS =====\n");
printf("Requests\tExecution Time (sec)\n");

for (j = 0; j < testCases; j++)
{
    int total = testSizes[j];

    clock_t startTime = clock();

    for (i = 0; i < total; i++)
    {
        Request test;

        test.id = i + 1;

        strcpy(test.source, "Main Warehouse");
        strcpy(test.destination, "Emergency Camp");
        strcpy(test.item, "Water");

        test.quantity = 50 + (i % 100);
        test.priority = 40 + (i % 61);

        addRequest(test);
    }

    while (heapCount > 0)
    {
        getTopRequest();
    }

    clock_t endTime = clock();

    double elapsed =
        (double)(endTime - startTime) /
        CLOCKS_PER_SEC;

    printf("%d\t\t%.6f\n", total, elapsed);
}
}

/* ===== MAIN FUNCTION ===== */

int main()
{
    int warehouse;

```



```

int hospital;
int centre;
int campA;
int campB;
int campC;

printf("\n=====\\n");
printf("  DISASTER RELIEF SUPPLY SYSTEM\\n");
printf("  GRAPH + DIJKSTRA + MAX HEAP\\n");
printf("=====\\n");

/* Create locations */

warehouse = createLocation("Main Warehouse");
hospital = createLocation("Emergency Hospital");
centre = createLocation("Relief Centre");
campA = createLocation("Emergency Camp");
campB = createLocation("Flood Area");
campC = createLocation("Medical Camp");

/* Create transportation network */

connectLocations(warehouse, centre, 6);
connectLocations(warehouse, hospital, 9);
connectLocations(centre, campA, 8);
connectLocations(centre, campB, 12);
connectLocations(hospital, campB, 5);
connectLocations(hospital, campC, 7);
connectLocations(campA, campC, 10);
connectLocations(campB, campC, 4);

showNetwork();

/* Relief requests */

Request requests[] =
{
    {101, "Main Warehouse", "Emergency Camp",
     "Food Packets", 600, 80},

    {102, "Main Warehouse", "Flood Area",
     "Drinking Water", 900, 95},

    {103, "Main Warehouse", "Emergency Hospital",
     "Medicines", 250, 100},

```

```

{104, "Main Warehouse", "Medical Camp",
"Medical Kits", 180, 98},

{105, "Main Warehouse", "Relief Centre",
"Blankets", 350, 65},

{106, "Main Warehouse", "Flood Area",
"Water Bottles", 500, 88}
};

int requestTotal =
    sizeof(requests) / sizeof(requests[0]);

/* Insert requests */

printf("\n===== ADDING RELIEF REQUESTS =====\n");

for (int i = 0; i < requestTotal; i++)
{
    printf("Request %d | Destination: %s | "
        "Priority: %d\n",
        requests[i].id,
        requests[i].destination,
        requests[i].priority);

    addRequest(requests[i]);
}

showRequests();

/* Process all requests */

dispatchSupplies();

/* Performance testing */

runPerformanceTest();

/* Final report */

printf("\n===== \n");
printf("        FINAL REPORT\n");
printf("===== \n");

```

```

printf("Locations Managed   : %d\n", nodeCount);
printf("Requests Processed  : %d\n", requestTotal);
printf("Graph Type          : Weighted Adjacency List\n");
printf("Route Method         : Dijkstra Algorithm\n");
printf("Priority Structure     : Max Heap\n");
printf("Queue Type             : Urgency Based\n");
printf("Time Measurement       : clock()\n");
printf("System Status          : COMPLETED\n");

printf("=====\\n");

return 0;
}

```

```

=====
DISASTER RELIEF SUPPLY SYSTEM
GRAPH + DIJKSTRA + MAX HEAP
=====

===== TRANSPORTATION NETWORK =====
Main Warehouse : Emergency Hospital [9 km] -> Relief Centre [6 km]
Emergency Hospital : Medical Camp [7 km] -> Flood Area [5 km] -> Main Warehouse [9 km]
Relief Centre : Flood Area [12 km] -> Emergency Camp [8 km] -> Main Warehouse [6 km]
Emergency Camp : Medical Camp [10 km] -> Relief Centre [8 km]
Flood Area : Medical Camp [4 km] -> Emergency Hospital [5 km] -> Relief Centre [12 km]
Medical Camp : Flood Area [4 km] -> Emergency Camp [10 km] -> Emergency Hospital [7 km]

===== ADDING RELIEF REQUESTS =====
Request 101 | Destination: Emergency Camp | Priority: 80
Request 102 | Destination: Flood Area | Priority: 95
Request 103 | Destination: Emergency Hospital | Priority: 100
Request 104 | Destination: Medical Camp | Priority: 98
Request 105 | Destination: Relief Centre | Priority: 65
Request 106 | Destination: Flood Area | Priority: 88

===== PRIORITY QUEUE =====
ID: 103 | Destination: Emergency Hospital | Supply: Medicines | Quantity: 250 | Priority: 100
ID: 104 | Destination: Medical Camp | Supply: Medical Kits | Quantity: 180 | Priority: 98
ID: 102 | Destination: Flood Area | Supply: Drinking Water | Quantity: 900 | Priority: 95
ID: 101 | Destination: Emergency Camp | Supply: Food Packets | Quantity: 600 | Priority: 80
ID: 105 | Destination: Relief Centre | Supply: Blankets | Quantity: 350 | Priority: 65
ID: 106 | Destination: Flood Area | Supply: Water Bottles | Quantity: 500 | Priority: 88

===== RELIEF DISPATCH =====

Supply          : Blankets
Quantity        : 350 units
Urgency         : 65
Finding shortest route...
Route: Main Warehouse -> Relief Centre
Travel Distance: 6 km
Status          : SUPPLY DELIVERED

All relief requests completed successfully.

===== PERFORMANCE ANALYSIS =====
Requests      Execution Time (sec)
10            0.000003
25            0.000014
50            0.000018
75            0.000046
100           0.000029

=====
FINAL REPORT
=====
Locations Managed : 6
Requests Processed : 6
Graph Type        : Weighted Adjacency List
Route Method      : Dijkstra Algorithm
Priority Structure : Max Heap
Queue Type        : Urgency Based
Time Measurement  : clock()
System Status     : COMPLETED
=====

```

```

Processing Request : 103
From      : Main Warehouse
To        : Emergency Hospital
Supply    : Medicines
Quantity  : 250 units
Urgency   : 100
Finding shortest route...
Route: Main Warehouse -> Emergency Hospital
Travel Distance: 9 km
Status    : SUPPLY DELIVERED

Processing Request : 104
From      : Main Warehouse
To        : Medical Camp
Supply    : Medical Kits
Quantity  : 180 units
Urgency   : 98
Finding shortest route...
Route: Main Warehouse -> Emergency Hospital -> Medical Camp
Travel Distance: 16 km
Status    : SUPPLY DELIVERED

Processing Request : 102
From      : Main Warehouse
To        : Flood Area
Supply    : Drinking Water
Quantity  : 900 units
Urgency   : 95
Finding shortest route...
Route: Main Warehouse -> Emergency Hospital -> Flood Area
Travel Distance: 14 km

```

Fig 2: Output Screen Shot

Use of Modern Tools

Tool	Purpose
C / GCC	Writing, compiling, and testing the program
Visual Studio Code	Editing and running the C source code
GitHub	Maintaining versions and submitting the project
Terminal	Compiling and executing the program
clock()	Measuring program execution time

The C program can be compiled with GCC and executed through the terminal. Screenshots of the source code, compilation process, program execution, and output can be included as evidence of tool usage.

3. Results and Validation

Sample Input

Destination	Supply Type	Quantity	Urgency
Flood Area B	Medicines	200	100
Emergency Hospital	Medical Kits	150	98
Affected Area C	Food Packets	700	75
Relief Centre	Blankets	300	65

Expected Priority Processing

Rank	Request ID	Destination	Urgency
1	3	Emergency Hospital	100
2	2	Flood Area B	98
3	6	Flood Area B	92
4	1	Affected Area A	85
5	4	Affected Area C	75
6	5	Relief Centre	65

Performance Test

Number of Requests	Measured Time (sec)
10	Record from program
20	Record from program
40	Record from program
60	Record from program
80	Record from program

The execution-time values should be taken directly from the program output. The test shows how processing time changes when the number of relief requests increases.

4. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear Route Search	Easy to implement	Less efficient for larger networks
Adjacency List Graph	Saves memory and efficiently stores connections	Performance depends on network structure

Approach	Advantage	Limitation
Weighted Graph + Dijkstra	Finds shortest-distance routes between locations	Requires additional route calculations
Max Heap	Provides quick access to the most urgent request	Does not keep all requests completely sorted
Priority Queue	Processes critical requests before lower-priority ones	Lower-priority requests may wait longer

The graph-based approach was selected to represent the transportation network efficiently. An adjacency list reduces unnecessary memory usage for connected locations, while Dijkstra's algorithm helps identify shorter routes. A max heap was chosen to quickly retrieve the most urgent relief request. Combining these structures provides efficient route management and priority-based supply distribution.

5. Broader Considerations

- **Societal Impact:**

The system can support faster distribution of essential supplies to disaster-affected areas. Processing requests according to urgency helps critical locations receive assistance earlier.

- **Environmental Impact:**

Selecting shorter routes can reduce unnecessary travel distance and fuel usage. This can make relief transportation more efficient.

- **Ethical Considerations:**

Urgency scores should be assigned accurately and fairly. Relief supplies should be distributed according to genuine emergency needs without discrimination.

6. Conclusion

The proposed system provides an efficient approach for managing disaster-relief supply distribution using graphs, heaps, and priority queues. The weighted graph represents the transportation network and supports shortest-route identification. The max heap gives priority to high-urgency relief requests over lower-priority requests. Together, these data structures improve route planning, request handling, and supply distribution. Overall, the system offers a practical and scalable solution for organized disaster-relief operations.

7. Student Reflection

Through this assignment, I learned how graphs can be applied to represent real-world transportation networks.

I gained an understanding of how heaps and priority queues can be used to arrange emergency requests according to their urgency. Implementing Dijkstra's algorithm also helped me understand shortest-path calculation and route selection.

I also improved my C programming, debugging, problem-solving, and performance-analysis skills.

This project helped me understand how different data structures can be combined to solve practical disaster-management problems.

8. References

- Roushan, A., Das, A., Dutta, A., & Bera, U. K., "A Multi-Objective Supply Chain Model for Disaster Relief Optimization Using Neutrosophic Programming and Blockchain-Based Smart Contracts," *Supply Chain Analytics*, 2025.
- Passacantando, M., & Raciti, F., "A Three-Stage Game Model of the Supply Chain in Disaster Relief Operations," *Journal of Optimization Theory and Applications*, Springer, 2025.
- Cook, R. A., & Lodree, E. J., "Vehicle Dispatching Policies for Last Mile Distribution in a Disaster Relief Supply Chain Network," *Annals of Operations Research*, Springer, 2025.
- Ahmed, H. N., Maoua, Z., Ahmed, S., & Ali, S. M., "Humanitarian Relief Supply Chain Performance Measurement: A Framework and Validation," *Journal of Industrial Information Integration*, 2025.
- Singh, J., Jaggi, C. K., & Kamna, K. M., "A Model for Relief Supply Chain Optimisation in a MultiProduct Distribution Environment," *International Journal of Logistics Systems and Management*, 2025.
- Arismawan, M., Short, M., Al-Mhdawi, M. K. S., Sentia, P. D., & Sari, C. M. A., "Humanitarian Logistics Supply Chain Management Framework: A Literature Review," *IOP Conference Series: Earth and Environmental Science*, 2025.
- Yılmaz, Ö. F., Guan, Y., & Yılmaz, B. G., "Designing a Resilient Humanitarian Supply Chain by Considering Viability Under Uncertainty: A Machine Learning Embedded Approach," *Transportation Research Part E: Logistics and Transportation Review*, 2025.
- Nawazish, M., & Padhi, S. S., "Designing Integrated Relief Aid Procurement and Last-Mile Distribution Strategies for Disaster Response Operations," *Transportation Research Part E: Logistics and Transportation Review*, 2025.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom'	Marks
Problem understanding &	CO2	PO1,	L4 – A	10
Application of course/don	CO2,	PO1,	L4 – A	20
Solution methodology / de	CO3,	PO3,	L5 – E	20
Use of appropriate moder	CO4	PO5,	L5 – E	10
Results, testing & validati	CO4,	PO4,	L4 – A	15
Analysis, trade-offs & jus	CO3,	PO2,	L5 – E	15
Broader considerations / I	CO5	PO6,	L5 – E	5
Technical documentation	CO5	PO9,	L5 – E	5
TOTAL	—	—	—	100

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis: Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____